Controlling Transactions and Locks in SQL Server

By Don Schlichting

Introduction

In the preceding articles of this series, Lock Granularity, Transactions, and ACID were introduced. Common lock types, such as Shared, Exclusive, and Update were explored, as well as using SP Lock to obtain current system lock information. In this article, the normal internal SQL locking methods will be manipulated using Lock Hints in order to obtain finer lock control.

Why?

Microsoft SQL does a very good job of self-monitoring and self-adjusting lock control. The engine is smart enough to change from row locks to page locks if the number of records involved increases to a point where individual row locks would no longer be optimal. It will also select a lock type specific to the type of statement being issued. So why bother with manual lock manipulation? There are some common situations where fine lock control will be desirable. One is a long running batch job where you as the developer or DBA, know that the lock you will need at the end is not a lock type SQL could have guessed looking at the beginning statements. The other situation is where the SQL server is doing simultaneous dual roles, such as receiving order entry input and doing reporting at the same time. At low transaction counts, this isn't a problem, but as the number of users and transactions increase, performance degradation occurs. This is especially common in real time production systems, where in addition to typical OLTP (Online Transaction Processing) and OLAP (Online Analytical Processing), the SQL server is issuing control instructions to some type of machine where performance hesitation cannot be tolerated. At the same time, it is unacceptable to have long delays in executive reporting. Ideally, these roles would be split between several different physical machines, but that is not always achievable due to labor or cost restraints. In these situations, manual lock manipulation may solve some of the issues.

Lock Hints

By default, SQL Server employ a lock type called Read Committed. The purpose of this lock is to ensure that any record set returned by a select only contains committed, accurate, real data. An example of non-real data, or inaccurate data would be if a user began a transaction to change an employee name from Mike to Jim, while this transaction is running, but before it commits, a second user issues a Select on employee name. Because the first transaction is not committed, SQL does not know what the real value of employee should be. Is it Mike, or Jim? It will be Jim if a commit is issued, but Mike if a rollback occurs. Rather than guess at a response, SQL's Read Committed says only committed data will be returned, therefore, the select statement will be blocked until the first transaction either commits or rollbacks. This type of action is also known as "Writers always block Readers." It ensures only accurate data is returned. The downside is the delay encountered by the select statement. In our example, the delay would be extremely brief, but if the database were large and our update affecting many rows, the delay may be unacceptable. Especially if the select is for some type of real time report that users are expecting to be returned almost instantaneously.

One of the first questions to ask in the above scenario is whether the data being returned by the select needs to be one hundred percent accurate. Many real time reports that display running counts, such as "widgets made this hour," need only to be approximate. The nature of the report is to provide a quick count and will be refreshed in a short amount of time. In this situation, there is an easy solution in SQL called Read Uncommitted.

NOLOCK

Read Uncommitted, also known as No Lock, is the opposite of the Read Committed lock described above. With No Lock, all rows are read, regardless of any exclusive locks that may be placed on them. In the above employee name example, it is possible to receive either name back, Mike or Jim back, depending on the timing of the select, commit, and rollback. The benefit of this is a very fast read that will not be blocked by exclusive locks. For the real time report example, it is a perfect fit. NO LOCK is enabled using the keyword WITH.

```
SELECT SUM(qty)AS RunningQty
FROM sales WITH (NOLOCK)
```

No Lock also has secondary benefit. Not only does it ignore Exclusive locks on rows, but it does not issue a Shared Lock on the records it reads. Therefore, it will not delay or block a transaction trying to write, a situation known as "Readers block writers." This can be very important in dual role systems where the database is responsible for simultaneous reporting and data input.

READPAST

Another similar option is called READPAST. With this hint, any rows being locked will be skipped. It is specified using a WITH command like NOLOCK.

```
SELECT au_lname
FROM authors WITH (READPAST)
```

Unlike NOLOCK, READPAST will skip only row locks, so if a long running transaction has a page lock, a select issued with READPAST will be blocked until the blocking transaction frees. In addition, READPAST will issue a Shared lock, so this reader will block writers.

Other Hints

There are thirteen different hints and isolation levels. Many are designed for the opposite purpose discussed in this article; they are used to issue blocking locks in order to guarantee a long running transaction has consistent data for the life of a batch.

For a complete list of Lock hints,
see http://msdn.microsoft.com/library/default.asp?url=/library/en-us/acdata/ac_8_con_7a_1hf7.asp .